CPSC 304 Project Cover Page

Milestone #: 4

Date: 04/02/26

Group Number: 36

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Julian Wright	84895358	w6i9k	jwrigh13@student.ubc.ca
Dorson Tang	94262474	d5v6x	dtang08@student.ubc.ca
Krishna Garcha	40548323	g8a8b	garchakri@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

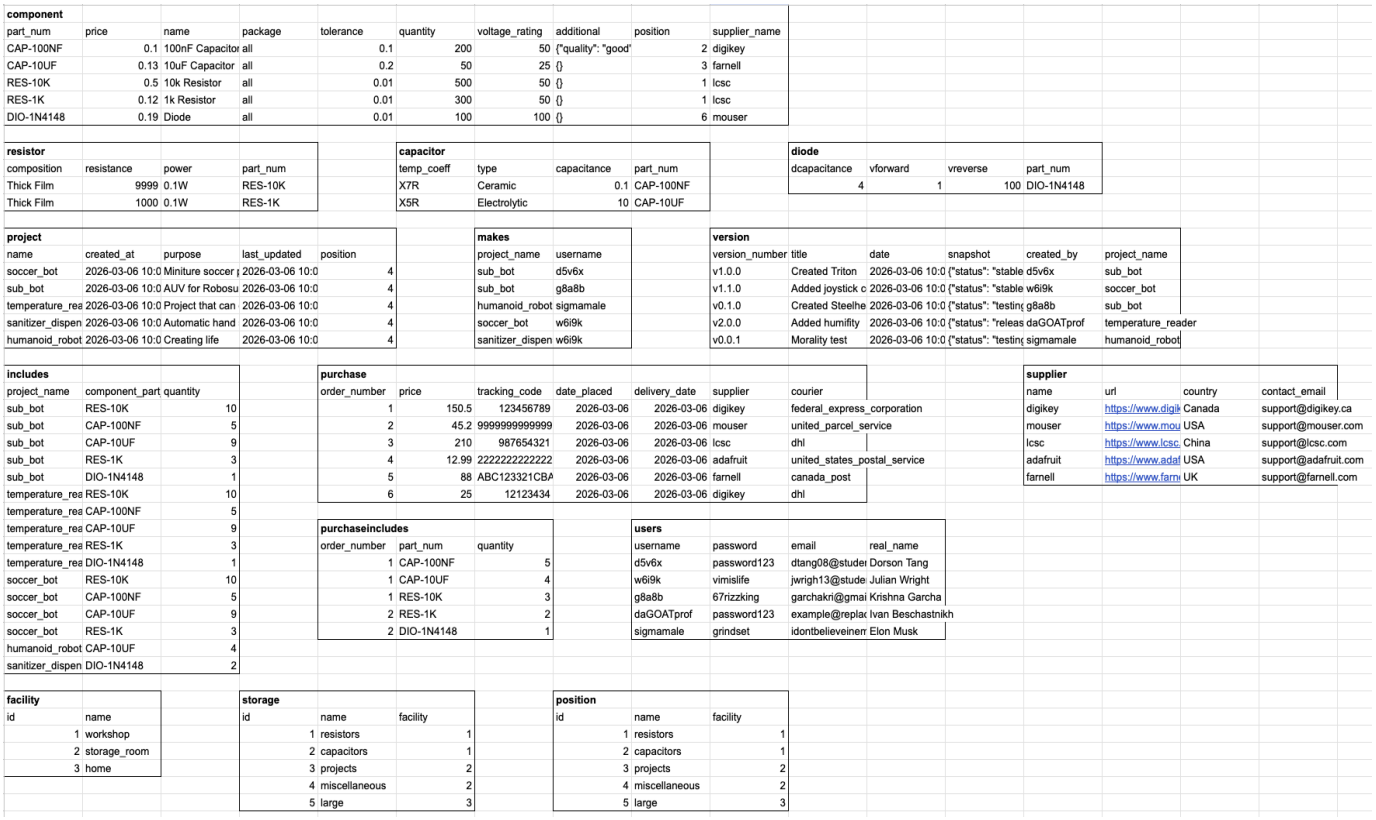
In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

SQL Script

The SQL that drops existing tables and then creates and populates all of the relations is in create_data.sql. It also provides the full database schema and clearly lists all keys (primary and foreign) and constraints.

The final version of the schema differs from our originally submitted one in that the representation of locations is different. Rather than including a Location relation with each type of location as an attribute (i.e., facility, storage_name, position) we switched to splitting those three up into different relations, that each include a foreign key to the parent (e.g., position -> storage_name, storage_name -> facility). With this representation, it's possible to have facilities that don't yet contain storages and storages that don't contain positions, which both makes more sense intuitively (and is a valid situation that could occur) and makes the process of creating new locations much easier. We also changed the ON UPDATE/ON DELETE behavior of various relations to improve user experience (e.g., on delete cascade for PurchaseIncludes so that a purchase can still be deleted even if it contains components).

Visual Representation



component

part_num	price	name	package	tolerance	quantity	voltage_rating	additional	position	supplier_name
CAP-100NF	0.1	100nF Capacitor	all	0.1	200	50	{"quality": "good"	2	digikey
CAP-10UF	0.13	10uF Capacitor	all	0.2	50	25	{}	3	farnell
RES-10K	0.5	10k Resistor	all	0.01	500	50	{}	1	lcsc
RES-1K	0.12	1k Resistor	all	0.01	300	50	{}	1	lcsc
DIO-1N4148	0.19	Diode	all	0.01	100	100	{}	6	mouser

resistor

composition	resistance	power	part_num
Thick Film	9999	0.1W	RES-10K
Thick Film	1000	0.1W	RES-1K

capacitor

temp_coeff	type	capacitance	part_num
X7R	Ceramic	0.1	CAP-100NF
X5R	Electrolytic	10	CAP-10UF

diode

dcapacitance	vforward	vreverse	part_num
4	1	100	DIO-1N4148

project

name	created_at	purpose	last_updated	position
soccer_bot	2026-03-06 10:0	Miniture soccer p	2026-03-06 10:0	4
sub_bot	2026-03-06 10:0	AUV for Robosu	2026-03-06 10:0	4
temperature_rea	2026-03-06 10:0	Project that can	2026-03-06 10:0	4
sanitizer_dispen	2026-03-06 10:0	Automatic hand	2026-03-06 10:0	4
humanoid_robot	2026-03-06 10:0	Creating life	2026-03-06 10:0	4

makes

project_name	username
sub_bot	d5v6x
sub_bot	g8a8b
humanoid_robot	sigmamale
soccer_bot	w6i9k
sanitizer_dispen	w6i9k

version

version_number	title	date	snapshot	created_by	project_name
v1.0.0	Created Triton	2026-03-06 10:0	{"status": "stable	d5v6x	sub_bot
v1.1.0	Added joystick c	2026-03-06 10:0	{"status": "stable	w6i9k	soccer_bot
v0.1.0	Created Steelhe	2026-03-06 10:0	{"status": "testing	g8a8b	sub_bot
v2.0.0	Added humifity	2026-03-06 10:0	{"status": "releas	daGOATprof	temperature_reader
v0.0.1	Morality test	2026-03-06 10:0	{"status": "testing	sigmamale	humanoid_robot

includes

project_name	component_part	quantity
sub_bot	RES-10K	10
sub_bot	CAP-100NF	5
sub_bot	CAP-10UF	9
sub_bot	RES-1K	3
sub_bot	DIO-1N4148	1
temperature_rea	RES-10K	10
temperature_rea	CAP-100NF	5
temperature_rea	CAP-10UF	9
temperature_rea	RES-1K	3
temperature_rea	DIO-1N4148	1
soccer_bot	RES-10K	10
soccer_bot	CAP-100NF	5
soccer_bot	CAP-10UF	9
soccer_bot	RES-1K	3
humanoid_robot	CAP-10UF	4
sanitizer_dispen	DIO-1N4148	2

purchase

order_number	price	tracking_code	date_placed	delivery_date	supplier	courier
1	150.5	123456789	2026-03-06	2026-03-06	digikey	federal_express_corporation
2	45.2	9999999999999	2026-03-06	2026-03-06	mouser	united_parcel_service
3	210	987654321	2026-03-06	2026-03-06	lcsc	dhl
4	12.99	2222222222222	2026-03-06	2026-03-06	adafruit	united_states_postal_service
5	88	ABC123321CBA	2026-03-06	2026-03-06	farnell	canada_post
6	25	12123434	2026-03-06	2026-03-06	digikey	dhl

supplier

name	url	country	contact_email
digikey	https://www.digik	Canada	support@digikey.ca
mouser	https://www.mou	USA	support@mouser.com
lcsc	https://www.lcsc	China	support@lcsc.com
adafruit	https://www.adaf	USA	support@adafruit.com
farnell	https://www.farn	UK	support@farnell.com

purchaseincludes

order_number	part_num	quantity
1	CAP-100NF	5
1	CAP-10UF	4
1	RES-10K	3
2	RES-1K	2
2	DIO-1N4148	1

users

username	password	email	real_name
d5v6x	password123	dtang08@studer	Dorson Tang
w6i9k	vimislife	jwrigh13@studer	Julian Wright
g8a8b	67rizzking	garchakri@gmai	Krishna Garcha
daGOATprof	password123	example@replac	Ivan Beschastnikh
sigmamale	grindset	idontbelieveinem	Elon Musk

facility

id	name
1	workshop
2	storage_room
3	home

storage

id	name	facility
1	resistors	1
2	capacitors	1
3	projects	2
4	miscellaneous	2
5	large	3

position

id	name	facility
1	resistors	1
2	capacitors	1
3	projects	2
4	miscellaneous	2
5	large	3

Final Project Description

Our final project is an electronics project management. It allows users to create projects and add electronic components (such as resistors, capacitors, etc.) for organizational/ tracking purposes. They can also choose suppliers for the components and track orders from those suppliers. Users can add locations (e.g., drawers, component bins, etc.) to both their components and projects to be able to find them better, something especially important for the components as they can get lost easily. We were able to achieve a cohesive application that can track all necessary information about components, and it has a very appealing UI for non technical users.

Implemented Queries

Projection

- Found in location.ts:38 in the backend, and locations.tsx:168 in the frontend

DELETE

- Component:
  - Backend: components.ts:214
  - Frontend: ComponentsTable.tsx:312, CapacitorTable.tsx:33, ResistorTable.tsx:33, DiodeTable.tsx:33
- Supplier:
  - Backend: shipping.ts:74
  - Frontend: SuppliersTable.tsx:124
- Courier:
  - Backend: shipping.ts:151
  - Frontend: CouriersTable.tsx:124
- Purchase:
  - Backend: shipping.ts265
  - Frontend: PurchasesTable.tsx:183

Join

- Found in location.ts:35,38 for the backend join and WHERE, and locations.tsx:121 for the frontend interface

INSERT

- Suppliers:
    - Backend: shipping.ts23
    - Frontend: SuppliersTable.tsx:99
- Couriers:
    - Backend: shipping.ts:101
    - Frontend: CouriersTable.tsx:97
- Purchases:
    - Backend: shipping.ts:203
    - Frontend: PurchasesTable.tsx:155

UPDATE:

- Suppliers:
    - Backend: shipping.ts:55
    - Frontend: SuppliersTable.tsx:105
- Couriers:
    - Backend: shipping.ts:134
    - Frontend: CouriersTable.tsx:104
- Purchases:
    - Backend: shipping.ts:246
    - Frontend: PurchasesTable.tsx:164

Update

- Locations:
    - Backend: location.ts:91 and location.ts:123 for creating and renaming locations
    - Frontend
        - Renaming: Locations.tsx:283, Locations.tsx:310, Locations.tsx:337
        - Creating: Locations.tsx:231

Select

- Found in component.ts:26 in the backend, and components.tsx:109 in the frontend.

Division

- Found in project.ts:12 in the backend, and projects.tsx:11 in the frontend
- At a high level, it returns all of the projects that are in an “includes” relation with every component

Aggregation with GROUP BY

- Get the total value (price * quantity) of elements per facility
- Backend: insights.ts:6
- Frontend: home.tsx:38 (fetch) and home.tsx:64 (render)

Aggregation with HAVING

- Find all projects with a cost (price * quantity per component in project) greater than some user-provided threshold
- Backend: insights.ts:28
- Frontend: home.tsx:53 (fetch) and home.tsx:105 (render)

Nested aggregation with GROUP BY

- Related query code found in:
    - Backend: shipping.ts:160
    - Frontend: PurchasesTable.tsx:155
- The purpose of the query is to allow a user to see purchases that fit within their specified budget.
- Explanation:
    - The query joins every component with the purchasesincludes relation that includes it, and computes the total price of the purchase.
    - The subquery’s HAVING clause is where we filter by the user’s budget preference. When grouping by the purchase order number, the nested query then filters for purchases that have a price less than the parsed user value.
    - Following this, a nested subquery counts how many such rows exist in the purchaseincludes relation and includes them in the count if they contain at least 1 component.

AI Tool Use Disclaimer

No AI tools were used for this milestone.